

PolicyGraph — Labeling Guide

Interactive tool for labeling scraped IAM policies to expand the training dataset.

Overview

The labeling workflow has three phases:

1. **Batch Scoring** — The trained GAT model + heuristics score all 959 scraped policies (automated, run overnight)
2. **Interactive Review** — A human reviews each policy in a terminal UI, accepting/rejecting/editing predictions (~30 sec/policy)
3. **Dataset Merge** — Reviewed policies are merged with the original 108-policy dataset for retraining

Progress is saved automatically so review can span multiple days.

Quick Start

```
# 1. Score all scraped policies with the trained model
python scripts/label_scraped_policies.py \
  --scraped data/scraped_policies \
  --out data/labeled_policies \
  --model checkpoints/long/best_model.pt \
  --score-only

# 2. Interactively review policies (quit anytime with 'q', resume later)
python scripts/label_scraped_policies.py \
  --scraped data/scraped_policies \
  --out data/labeled_policies \
  --review-only

# 3. Merge labeled policies with original dataset
python scripts/merge_labeled_datasets.py \
  --original data/raw/samples \
  --new data/labeled_policies \
  --output data/raw/samples_expanded

# 4. Retrain on expanded dataset
policygraph train --data-dir data/raw/samples_expanded
```

Commands Reference

Score-Only Mode (Batch Overnight)

Scores all policies without human interaction. Safe to run on a server overnight.

```
python scripts/label_scraped_policies.py \
  --scraped data/scraped_policies \
  --out data/labeled_policies \
  --model checkpoints/long/best_model.pt \
  --score-only
```

What it does:

- Loads the trained GAT model from checkpoint
- For each scraped `.tf` file:
- Extracts IAM policy JSON (if possible)
- Runs the policy through the model → gets risk score (0-1)
- Falls back to heuristic scoring if JSON extraction fails
- Saves all scores to `data/labeled_policies/model_scores.json`
- Reports distribution (vulnerable / secure / unknown)

Expected time: ~2-3 minutes for 959 policies (most time is model loading)

Interactive Review

Presents each policy in a terminal UI for human review.

```
python scripts/label_scraped_policies.py \
  --scraped data/scraped_policies \
  --out data/labeled_policies \
  --review-only
```

Keyboard shortcuts:

Key	Action	Description
a	Accept	Accept the model's prediction as-is
r	Reject	Flip the label (vulnerable → secure, or vice versa)
e	Edit	Cycle severity: critical → high → medium → low
s	Skip	Skip this policy (can review later)
q	Quit	Save progress and exit

The UI shows:

- Progress bar (e.g., `[42/959]`)
- Statistics (reviewed, vulnerable, secure, remaining)
- Review rate and ETA
- Risk score (color-coded: ● high, ● medium, ● low)
- Model prediction and vulnerability type
- Policy content preview (first 25 lines)

Full Pipeline (Score + Review)

Runs both scoring and review in one command.

```
python scripts/label_scraped_policies.py \
  --scraped data/scraped_policies \
  --out data/labeled_policies \
  --model checkpoints/long/best_model.pt
```

Limit to N Policies (Testing)

```
python scripts/label_scraped_policies.py \
  --scraped data/scraped_policies \
  --out data/labeled_policies \
  --model checkpoints/long/best_model.pt \
  --limit 10
```

How to Resume from Saved Progress

Progress is automatically saved:

- Every 10 reviews (auto-save)
- When you press `q` to quit
- At the end of a review session

To resume, just run the same command again:

```
python scripts/label_scraped_policies.py \
  --scraped data/scraped_policies \
  --out data/labeled_policies \
  --review-only
```

The tool will:

1. Load existing scores from `model_scores.json`
2. Load review state from `labeling_state.json`
3. Skip already-reviewed policies
4. Resume from where you left off

Checking Progress

```
python -c "
import json
state = json.load(open('data/labeled_policies/labeling_state.json'))
print(f'Reviewed: {state[\"total_reviewed\"]}')
print(f'Scored: {state[\"total_scored\"]}')
print(f'Skipped: {len(state[\"skipped\"])}')
print(f'Last index: {state[\"last_index\"]}')
for session in state.get('session_history', []):
    print(f'  Session {session[\"timestamp\"]}: {session[\"reviewed\"]} reviewed in {session[\"elapsed_seconds\"]}s')
"
```

Best Practices for Labeling

What Makes a Policy “Vulnerable”?

A policy is **vulnerable** if it enables privilege escalation — meaning an attacker with access to a principal using this policy could gain higher privileges than intended.

Common vulnerability patterns:

Pattern	Severity	Example
<code>iam:PassRole</code> + compute service	Critical	PassRole + Lambda create → admin via function
<code>iam:*</code> (full IAM access)	Critical	Can create admin users, modify any policy
<code>Action: *</code> on <code>Resource: *</code>	Critical	Unrestricted access to everything
<code>iam:AttachUserPolicy</code>	Critical	Can attach AdministratorAccess to self
<code>iam:PutRolePolicy</code>	High	Can write inline policy granting any permission
<code>iam:CreatePolicyVersion</code>	Critical	Can create new version of any managed policy
<code>sts:AssumeRole</code> on <code>*</code>	High	Can assume any role including admin roles
<code>iam:CreateAccessKey</code>	High	Can create credentials for other users

What Makes a Policy “Secure”?

A policy is **secure** if it follows the principle of least privilege:

- **Specific actions** (not wildcards)
- **Scoped resources** (specific ARNs, not `*`)
- **Conditions** present (MFA, IP restrictions, tags)
- **Deny statements** with guardrails
- **Read-only** actions (Describe, List, Get*)

When to Skip

Skip a policy if:

- It's a role definition with no policy statements
- It's too fragmentary to evaluate
- The HCL is too complex to understand quickly

Severity Guidelines

Severity	Risk Score	Description
Critical	9-10	Direct path to admin access
High	7-8	Privilege escalation possible with 1-2 steps
Medium	5-6	Partial privilege escalation or broad access
Low	0-4	Follows least privilege, no escalation path

Output Files

After labeling, the `data/labeled_policies/` directory contains:

```
data/labeled_policies/
├── labeling_state.json      # Review progress (resume state)
├── model_scores.json       # Model predictions for all policies
├── LABELS.json            # Labels in PolicyGraph format
├── policies_labeled.csv    # Labels in CSV format
├── policies/              # Extracted policy JSON files
│   ├── scraped_escalation_passrole_lambda_abc123.json
│   ├── scraped_secure_def456.json
│   └── ...
```

LABELS.json format

Compatible with `data/raw/samples/LABELS.json` :

```
{
  "description": "Labels for scraped IAM policies",
  "version": "1.0.0",
  "total_policies": 42,
  "vulnerable_count": 15,
  "secure_count": 27,
  "labels": [
    {
      "filename": "scraped_escalation_passrole_lambda_abc123.json",
      "label": "vulnerable",
      "severity": "critical",
      "vulnerability_type": "passrole_lambda",
      "risk_score": 9,
      "attack_path": ["PassRole → Lambda → Admin"],
      "remediation": "Restrict PassRole to specific ARNs..."
    }
  ]
}
```

policies_labeled.csv format

Compatible with `data/raw/policies_labeled.csv` :

```
filename,label,vulnerability_type,severity,risk_patterns,...
scraped_escalation_passrole_lambda_abc123.json,vulnerable,passrole_lambda,critical,...
scraped_secure_def456.json,secure,none,low,...
```

Expected Time Estimates

Task	Time	Notes
Batch scoring (959 policies)	~2-3 min	Automated, no interaction needed
Interactive review (all 959)	~8 hours	At ~30 sec/policy
Review 100 policies (session)	~50 min	Manageable daily session
Dataset merge	< 1 min	Automated
Retraining	~5-10 min	Depends on dataset size

Recommended Schedule

- **Day 1:** Run batch scoring overnight
- **Days 2-9:** Review 100-120 policies per day (~1 hour sessions)
- **Day 10:** Merge datasets and retrain

Or prioritize high-confidence predictions:

- Review high-score (>0.7) policies first (likely vulnerable, need confirmation)
- Then review low-score (<0.2) policies (likely secure, quick accepts)
- Leave mid-range (0.2-0.7) for last (need more thought)